

```

!pip install -q transformers accelerate datasets torch codecarbon
_____ 0.0/380.8 kB ? eta -:--:--
_____ 380.8/380.8 kB 14.2 MB/s eta
0:00:00
_____ 0.0/155.6 kB ? eta -:--:--
_____ 155.6/155.6 kB 19.1 MB/s eta
0:00:00
_____ 8.0/8.0 MB 105.3 MB/s eta
0:00:00
_____ 3.2/3.2 MB 126.2 MB/s eta
0:00:00

!pip install -q --upgrade transformers
_____ 10.1/10.1 MB 45.4 MB/s eta
0:00:00

from huggingface_hub import login
login()

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(

{"model_id": "10acdd9863744d98bf0df4956eae4711", "version_major": 2, "vers
ion_minor": 0}

```

## Load Model

```

import os
os.environ["CUDA_LAUNCH_BLOCKING"] = "1"

import torch
import os
from transformers import AutoProcessor, AutoModelForImageTextToText

model_id = "google/gemma-3n-E4B-it"
os.makedirs("/content/offload", exist_ok=True)

processor = AutoProcessor.from_pretrained(model_id)
model = AutoModelForImageTextToText.from_pretrained(
    model_id,
    dtype=torch.bfloat16,

```

```

        device_map="auto",
        offload_folder="/content/offload"
    )
    model.eval()
    print("Ready.")

{"model_id": "45f9e53db3d340ab8d04fb240a4e05ba", "version_major": 2, "version_minor": 0}

{"model_id": "0339850f6ed34d6a92162bc7e417276d", "version_major": 2, "version_minor": 0}

{"model_id": "4e0640b32cc34931878f496bdadee399", "version_major": 2, "version_minor": 0}

{"model_id": "96a194dc239a4f7bad4a2606d9e39e51", "version_major": 2, "version_minor": 0}

{"model_id": "7136a239facf48a6977d53f70006f908", "version_major": 2, "version_minor": 0}

{"model_id": "5756e7db7df64806804e01c55c4db54e", "version_major": 2, "version_minor": 0}

{"model_id": "1aab9533756e4d3782914288b8c0284a", "version_major": 2, "version_minor": 0}

{"model_id": "f2c34ca1ecd24948be9c95dddc0a48f7", "version_major": 2, "version_minor": 0}

{"model_id": "c025d5d0dac84aebbe10bf3cefdfa54b", "version_major": 2, "version_minor": 0}

{"model_id": "4fe31acee19a4a2cbef59b6bbb4958b5", "version_major": 2, "version_minor": 0}

{"model_id": "412cb06080cb44de9f2b3b165589af22", "version_major": 2, "version_minor": 0}

{"model_id": "4ea2e32eb86a45258b366da316735105", "version_major": 2, "version_minor": 0}

WARNING:accelerate.big_modeling:Some parameters are on the meta device
because they were offloaded to the cpu.

Ready.

```

## Load VQAv2 Sample

```

from datasets import load_dataset
import random

```

```

ds = load_dataset("lmms-lab/VQAv2", split="validation",
streaming=True)

samples = []
for item in ds:
    samples.append(item)
    if len(samples) >= 200:
        break

yes_no = [s for s in samples if s["answer_type"] == "yes/no"][:4]
number = [s for s in samples if s["answer_type"] == "number"][:3]
other = [s for s in samples if s["answer_type"] == "other"][:3]
stratified = yes_no + number + other
random.shuffle(stratified)
print(f"Sample size: {len(stratified)}")

{"model_id": "ee58327fddd1470bbaafa2acd8da51af", "version_major": 2, "version_minor": 0}

{"model_id": "a7b0462a3ad040d4b9effcbff6cdfb45", "version_major": 2, "version_minor": 0}

{"model_id": "a93f867df5b54b098fa0f6fdf3e54039", "version_major": 2, "version_minor": 0}

{"model_id": "2676c8f7a8424ca9bd63b2ab1eaf11f6", "version_major": 2, "version_minor": 0}

{"model_id": "b8a5641cbdc445cc92a3560f0fcff9c0", "version_major": 2, "version_minor": 0}

{"model_id": "2a8757d653a14ff7b1c5cb82a215d8fa", "version_major": 2, "version_minor": 0}

{"model_id": "54356c72b65943c88d0600bca6d120b5", "version_major": 2, "version_minor": 0}

Sample size: 10

```

for 50 samples:

```

from datasets import load_dataset
import random

ds = load_dataset("lmms-lab/VQAv2", split="validation",
streaming=True)

samples = []
for item in ds:
    samples.append(item)
    if len(samples) >= 500:

```

```

        break

yes_no = [s for s in samples if s["answer_type"] == "yes/no"][:15]
number = [s for s in samples if s["answer_type"] == "number"][:15]
other = [s for s in samples if s["answer_type"] == "other"][:20]
stratified = yes_no + number + other
random.shuffle(stratified)
print(f"Sample size: {len(stratified)}")

{"model_id": "b4700fbf30674985aa69c1a7004fe24b", "version_major": 2, "version_minor": 0}

{"model_id": "a8ee7ec4d455475a894b3d71044a9897", "version_major": 2, "version_minor": 0}

{"model_id": "16482be5d2f14e45aaf29ef5a6a99250", "version_major": 2, "version_minor": 0}

{"model_id": "cc39de17c876493b964ed45dbb052b3f", "version_major": 2, "version_minor": 0}

{"model_id": "fcdf1934467f4c55ad9de02e053310e0", "version_major": 2, "version_minor": 0}

{"model_id": "0d689fb083294c978c249c426742fef0", "version_major": 2, "version_minor": 0}

Sample size: 50

```

## Inference + Logging

Gemma 4B

```

import time
from codecarbon import EmissionsTracker
import torch

tracker = EmissionsTracker(log_level="error")
tracker.start()

results = []

for i, item in enumerate(stratified):
    image = item["image"]
    question = item["question"]

    messages = [
        {
            "role": "user",
            "content": [
                {"type": "image", "image": image},

```

```

        {"type": "text", "text": f"Question: {question} Answer
briefly."}
    ]
    }
]

prompt = processor.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

inputs = processor(
    images=image,
    text=prompt,
    return_tensors="pt"
).to(model.device, dtype=torch.bfloat16)

start = time.perf_counter()
with torch.no_grad():
    output = model.generate(
        **inputs,
        max_new_tokens=20,
        do_sample=False
    )
ttft = time.perf_counter() - start

predicted = processor.decode(output[0],
skip_special_tokens=True).strip()
gt = item["multiple_choice_answer"]

results.append({
    "question": question,
    "predicted": predicted,
    "ground_truth": gt,
    "answer_type": item["answer_type"],
    "ttft_seconds": round(ttft, 4)
})

print(f"[{i+1}/50] Q: {question[:40]} | GT: {gt} | Pred:
{predicted[-40:]}")

emissions = tracker.stop()
print(f"\nDone. Emissions: {emissions:.6f} kg CO2")

[1/50] Q: What color are the walls? | GT: white | Pred:  the walls?
Answer briefly.
model
Beige.
[2/50] Q: How many beds? | GT: 1 | Pred: ow many beds? Answer briefly.

```

model  
One.  
[3/50] Q: What kind of room is this? | GT: bedroom | Pred: is this?  
Answer briefly.  
model  
Bedroom.  
[4/50] Q: How many are playing ball? | GT: 1 | Pred: playing ball?  
Answer briefly.  
model  
One.  
[5/50] Q: Is this rice noodle soup? | GT: yes | Pred: noodle soup?  
Answer briefly.  
model  
Yes.  
[6/50] Q: How many seats are there? | GT: 1 | Pred: ts are there?  
Answer briefly.  
model  
One.  
[7/50] Q: What is the child eating? | GT: donut | Pred: d eating?  
Answer briefly.  
model  
A donut.  
[8/50] Q: How many umbrellas do you see? | GT: 0 | Pred: \*\*two\*\*  
umbrellas visible in the image.  
[9/50] Q: How many people are on the field? | GT: 3 | Pred: There are  
\*\*three\*\* people on the field.  
[10/50] Q: How many frames are on the wall? | GT: 7 | Pred: odel  
There are \*\*6\*\* frames on the wall.  
[11/50] Q: What color is the kids hair? | GT: blonde | Pred: kids  
hair? Answer briefly.  
model  
Blonde.  
[12/50] Q: What is the chair made of? | GT: leather | Pred: be made  
of \*\*leather or faux leather\*\*.  
[13/50] Q: How many pictures on the wall? | GT: 7 | Pred: el  
There are \*\*6\*\* pictures on the wall.  
[14/50] Q: Is this room in someone's home? | GT: yes | Pred: meone's  
home? Answer briefly.  
model  
Yes.  
[15/50] Q: What is to the right of the soup? | GT: chopsticks | Pred:  
r briefly.  
model  
Chopsticks and a spoon.  
[16/50] Q: What website copyrighted the picture? | GT: foodiebakercom  
| Pred: er briefly.  
model  
<http://foodiebaker.com>  
[17/50] Q: How many pictures are there? | GT: 7 | Pred: briefly.

model

There are **\*\*6\*\*** pictures.

[18/50] Q: Which room is this? | GT: bedroom | Pred: is this? Answer briefly.

model

Bedroom.

[19/50] Q: Is the bed white? | GT: yes | Pred: the bed white? Answer briefly.

model

No.

[20/50] Q: How many photo's can you see? | GT: 1 | Pred: can you see? Answer briefly.

model

One.

[21/50] Q: Where is he looking? | GT: down | Pred: riefly.

model

He is looking to his left.

[22/50] Q: Is there a chain link fence in the image | GT: no | Pred: in the image? Answer briefly.

model

No.

[23/50] Q: Did the batter hit the ball? | GT: yes | Pred: hit the ball? Answer briefly.

model

No.

[24/50] Q: What is the man doing in the street? | GT: walking | Pred: n is crossing the street at a crosswalk.

[25/50] Q: Could this be a hotel room? | GT: no | Pred: a hotel room? Answer briefly.

model

No.

[26/50] Q: Is it daylight in this picture? | GT: yes | Pred: this picture? Answer briefly.

model

Yes.

[27/50] Q: How many skyscrapers are there? | GT: 2 | Pred: re are **\*\*two\*\*** skyscrapers in the image.

[28/50] Q: Is this room decorated for the 1970s? | GT: no | Pred: for the 1970s? Answer briefly.

model

No.

[29/50] Q: What is he on top of? | GT: picnic table | Pred: n top of? Answer briefly.

model

A bench.

[30/50] Q: Are these twin mattresses? | GT: no | Pred: in mattresses? Answer briefly.

model

No.

[31/50] Q: What is the size of the bed? | GT: queen | Pred: rmine the exact size of the bed. However

[32/50] Q: Is the boy playing baseball? | GT: yes | Pred: ing baseball? Answer briefly.

model

Yes.

[33/50] Q: What does the truck on the left sell? | GT: ice cream | Pred: ft sells \*\*sundaes, shakes, and cones\*\*.

[34/50] Q: Are the windows big? | GT: yes | Pred: windows big? Answer briefly.

model

Yes.

[35/50] Q: What is behind the foot of the bed? | GT: chair | Pred: f the bed? Answer briefly.

model

A desk.

[36/50] Q: Why is the cow laying down? | GT: tired | Pred: y.

model

The cow is laying down to rest.

[37/50] Q: What color is the grass in this picture? | GT: green | Pred: because the image is in black and white.

[38/50] Q: How many colors is the building on the r | GT: 1 | Pred: e right is painted \*\*one color\*\*: brown.

[39/50] Q: What animal print does that chair resemb | GT: none | Pred: l

The chair resembles \*\*leopard print\*\*.

[40/50] Q: How many signs? | GT: 4 | Pred: briefly.

model

There are \*\*four\*\* signs.

[41/50] Q: Are the lights on in this room? | GT: no | Pred: in this room? Answer briefly.

model

Yes.

[42/50] Q: How many chairs are in the photo? | GT: 1 | Pred: in the photo? Answer briefly.

model

One.

[43/50] Q: Is this a creamy soup? | GT: no | Pred: a creamy soup? Answer briefly.

model

No.

[44/50] Q: How many stories is the building on the | GT: 3 | Pred: e left appears to have \*\*five\*\* stories.

[45/50] Q: Is the bed made? | GT: yes | Pred: the bed made? Answer briefly.

model

No.

[46/50] Q: Why is there a gap between the roof and | GT: yes | Pred: ilation\*\*. It allows for airflow to help

[47/50] Q: Is that a folding chair? | GT: no | Pred: folding chair?  
 Answer briefly.  
 model  
 No.

[48/50] Q: What color is the bedspread? | GT: white | Pred:  
 bedspread? Answer briefly.  
 model  
 Beige.

[49/50] Q: What are the people in the background do | GT: watching |  
 Pred: tching the skateboarder perform a trick.

[50/50] Q: How many sources of light are there? | GT: 1 | Pred: e\*\*  
 primary source of light: the window.

Done. Emissions: 0.006200 kg CO2

## Baseline Results

```
import json

correct = sum(
    1 for r in results
    if r["ground_truth"].lower() in r["predicted"].lower()
)
vqa_score = correct / len(results)
avg_ttft = sum(r["ttft_seconds"] for r in results) / len(results)

baseline = {
    "model": "gemma-3n-E4B",
    "n_samples": len(results),
    "vqa_accuracy": round(vqa_score, 4),
    "avg_ttft_seconds": round(avg_ttft, 4),
    "total_kg_co2": round(emissions, 8),
    "kg_co2_per_sample": round(emissions / len(results), 10),
    "results": results
}

with open("baseline_results.json", "w") as f:
    json.dump(baseline, f, indent=2)

print(f"VQA Accuracy : {vqa_score:.2%}")
print(f"Avg TTFT      : {avg_ttft:.3f}s")
print(f"C02 total       : {emissions:.6f} kg")
print(f"C02/sample      : {emissions/len(results):.8f} kg")

VQA Accuracy : 36.00%
Avg TTFT      : 12.440s
C02 total     : 0.006200 kg
C02/sample    : 0.00012401 kg
```

```
from google.colab import files
files.download("baseline_results.json")
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
```